



GEM-LLM: Identifying contextual equivalent mutants via large language models; A global invariant-based approach

Mohammad Tanhaei^a*, Roohallah Alizadehsani^b

^a Department of Engineering, Ilam University, Ilam, Iran

^b Institute for Intelligent Systems Research and Innovation (IISRI) Deakin University, Waurn Ponds, VIC 3216, Australia

ARTICLE INFO

Dataset link: <https://github.com/tanhaei/GEM-LLM>

Keywords:

Mutation testing
Equivalent mutants
Large language models
Program invariants
Global analysis

ABSTRACT

Mutation testing represents a robust technique for assessing the adequacy of test suites. However, the challenge of equivalent mutants persists, hindering its widespread adoption in practical settings. These mutants, while syntactically distinct, are functionally identical to the original program, leading to inflated computational costs and inaccurate mutation scores. Traditional detection approaches often overlook "contextual equivalence," where mutants behave identically only under the broader constraints and domain-specific boundaries of the entire program.

To address this, we introduce GEM-LLM, a hybrid framework that leverages large language models for semantic invariant inference and SMT solvers for formal equivalence verification. GEM-LLM employs inter-procedural slicing to provide large language models with detailed calling-context information, enabling the identification of domain-specific constraints that traditional tools miss.

Our evaluation on 10 projects from the Defects4J benchmark demonstrates that GEM-LLM classifies 25%–30% of overlooked surviving mutants as equivalent, achieving a precision of 98%. A detailed breakdown reveals superior performance on relational and boundary operators, with detection rates up to 34.2%. By shifting from local to global analysis, GEM-LLM offers a scalable and formally rigorous solution to the equivalent mutant problem, enhancing the accuracy and reliability of mutation testing in large-scale enterprise software systems.

1. Introduction

Mutation testing stands as a cornerstone technique in software engineering, evaluating test suite effectiveness by injecting artificial faults, known as mutants, into the source code (Jia & Harman, 2011). A robust test suite detects these alterations, "killing" the mutants by revealing behavioral divergences from the original program. This approach uncovers subtle defects and improves overall test quality (Wang et al., 2024). Nevertheless, its broad application in enterprise environments faces significant obstacles, primarily the equivalent mutant problem (Kushigian et al., 2024).

Equivalent mutants are syntactically different from the original code yet functionally identical, rendering no test case capable of distinguishing them. This inflates computational overhead and diminishes mutation scores, eroding developer confidence in testing assessments. Although undecidable in general (Offutt & Pan, 1997), recent empirical studies indicate that equivalent mutants occur frequently, comprising up to 30% of generated mutants in real-world Java programs (Kushigian et al., 2024).

Existing approaches predominantly rely on local static or dynamic analyses for equivalence detection. Static methods, such as compiler optimizations or SMT solver-based proofs (Papadakis et al., 2015), excel at syntactic equivalence but fail to capture "contextual equivalence". These mutants appear distinct in isolation but align under global program invariants and domain constraints. For instance, a mutant altering behavior for negative inputs becomes equivalent if the calling context ensures positive values. Dynamic methods, like runtime invariant monitoring (Schuler & Zeller, 2009), mitigate some gaps but incur high runtime costs and remain bounded by test suite coverage.

Large language models have revolutionized code comprehension and logical reasoning, shifting from rule-based to semantic-driven analysis (Pei et al., 2023). These models demonstrate proficiency in inferring developer intent, identifying architectural patterns, and extracting invariants from code corpora (Liu et al., 2026). Recent integrations of large language models in mutation testing generate diverse mutants (Wang et al., 2024) and employ iterative debugging for fault replication (Straubinger et al., 2025). These advancements highlight

* Corresponding author.

E-mail addresses: m.tanhaei@ilam.ac.ir (M. Tanhaei), r.alizadehsani@deakin.edu.au (R. Alizadehsani).

their potential to tackle longstanding issues like the equivalent mutant problem. However, applying large language models for contextual equivalence detection — linking local mutations to global semantics — remains underexplored.

In this paper, we present GEM-LLM, a hybrid framework that combines large language model-derived global invariant inference with SMT solvers for formal equivalence verification. It utilizes inter-procedural slicing to furnish large language models with contextual call-graph perspectives. GEM-LLM identifies domain constraints that elude conventional tools. Our contributions are threefold:

- We pioneer large language model-driven global analysis in mutation testing, transcending method-level boundaries to encompass program-wide invariants.
- We develop a hybrid pipeline that integrates large language model generative capabilities with SMT-based validation, ensuring invariant robustness.
- Through empirical evaluation on the Defects4J dataset, we demonstrate GEM-LLM’s ability to identify up to 30% more equivalent mutants than local methods, with 98% precision.

The innovation of this article resides in its pioneering integration of semantic invariant inference from large language models with formal equivalence proofs via SMT solvers, enabling the detection of contextual equivalent mutants that traditional local analysis methods overlook. This hybrid approach not only enhances the precision and scalability of mutation testing but also bridges the gap between neural semantic reasoning and symbolic formal verification, providing a novel framework that advances the state-of-the-art in software testing by addressing longstanding challenges in equivalent mutant detection through global program analysis. Furthermore, this work extends beyond software engineering, offering insights into hybrid AI-formal methods that could inspire applications in interdisciplinary domains such as materials science and device simulation.

2. Related work

Detecting equivalent mutants has been a central challenge in mutation testing for decades. In this section, we categorize existing literature into traditional detection techniques and recent advances in large language model-based program analysis.

2.1. Traditional equivalent mutant detection

Conventional approaches to the equivalent mutant problem divide into static and dynamic analysis. Static methods frequently employ compiler optimizations to identify syntactic-level equivalence (Hierons et al., 1999; Offutt & Pan, 1997). For instance, Tip (1995) explored program slicing techniques to reduce the search space for equivalent mutants. More advanced methods utilize constraint-based testing and satisfiability modulo theories solvers, such as Z3, to prove semantic equivalence between code fragments (Papadakis et al., 2015). However, these techniques remain predominantly “local”, focusing on the mutated statement’s immediate control flow. They often miss global external constraints that could render a mutant equivalent in a specific project context.

Dynamic analysis relies on program execution. Schuler and Zeller (2009) proposed monitoring for “invariant violations” during runtime. If a mutant does not alter any observed program invariants, it is deemed likely equivalent. These methods are effective but computationally expensive and limited by the coverage of the existing test suite.

Recent studies have examined empirical patterns in mutants. For example, Hans et al. (2025) introduced mutation operators based on empirical fault patterns from orthogonal defect classification, reducing mutant quantity while increasing relevance to real defects. In industrial contexts, Brasil et al. (2024) reported experiences applying

mutation testing in fintech, highlighting its role in identifying test weaknesses beyond traditional coverage. For tool comparisons, Guerino et al. (2024) evaluated Python mutation tools, noting variations in equivalent mutant generation. Additionally, Kushigian et al. (2024) proposed equivalent mutant suppression for Java, achieving effective identification using AST patterns.

2.2. LLMs and program invariants

Large language models have transformed software engineering tasks. Recent studies indicate that large language models can effectively infer program invariants from source code (Pei et al., 2023). For example, Liu et al. (2026) demonstrated that large language models could generate loop invariants with higher accuracy than symbolic execution in certain domains. Furthermore, large language models have been applied to assist in bounded model checking, providing candidate invariants to prune the state space (Tian et al., 2024).

Further advancements include (Wang et al., 2024), which evaluated large language models for mutant generation, showing improved fault detection but elevated equivalence rates. Foster et al. (2025) presented a mutation-guided large language model system at Meta for privacy-aware testing. Straubinger et al. (2025) utilized iterative large language model-driven debugging for mutation testing, achieving superior detection over baselines. Despite these progressions, the application of large language models to detect “contextual equivalence” in mutation testing remains an under-explored area. Our work bridges this gap by integrating semantic inference with formal verification.

2.3. Summary of existing approaches

To better position our work, Table 1 provides a comparative analysis of traditional methods against our large language model-based global approach. We evaluate these methods across four dimensions: the scope of analysis (local vs. global), the underlying reasoning engine, the handling of invariants, and the level of automation.

As illustrated in Table 1, traditional static and dynamic methods confine their focus to the immediate vicinity of the mutation. In contrast, our approach harnesses the semantic reasoning capabilities of large language models to bridge the divide between local code alterations and global program invariants, providing a more comprehensive detection mechanism.

3. Methodology

This section describes the proposed GEM-LLM framework for detecting contextual equivalent mutants. The framework employs large language models as a high-level reasoning engine for invariant generation and verifies them through formal methods. The complete workflow of our approach is depicted in Fig. 1.

As shown in the figure, the workflow commences upon a mutant surviving the initial test suite execution. It proceeds through three primary phases: inter-procedural slicing to capture external context, large language model-based invariant generation utilizing chain-of-thought prompting, and hybrid formal verification with SMT solvers.

3.1. Formal foundation of contextual equivalence

Let F denote the function containing the mutation, and C represent the calling context of F within the system. We define the input domain of F as D . Traditional equivalent mutant detection considers the entire domain D . However, our approach restricts this domain to a feasible subset $D_{context} \subseteq D$ defined by global invariants I_G .

Formally, a mutant M is contextually equivalent if:

$$\forall x \in D : I_G(x) \implies \llbracket P \rrbracket(x) = \llbracket M \rrbracket(x) \quad (1)$$

where $\llbracket \cdot \rrbracket$ denotes the semantic denotation of the program. The challenge lies in extracting I_G and verifying the implication.

Table 1

Comparison of state-of-the-art equivalent mutant detection techniques, highlighting the scope of analysis (local vs. global), reasoning mechanisms (syntactic, symbolic, semantic), invariant handling (none, partial, rich), and automation levels (moderate, high, fully automated).

Approach	Scope	Reasoning engine	Invariants	Automation
Compiler-based (Papadakis et al., 2015)	Local	Syntactic/CFG	No	Fully Automated
Constraint-based (Offutt & Pan, 1997)	Local	SMT/Symbolic	Partial	Moderate
Dynamic Detection (Schuler & Zeller, 2009)	Trace	Runtime Traces	Empirical	Fully Automated
Search-based (Harman et al., 2012)	Global	Meta-heuristics	No	High
Ours (GEM-LLM)	Global	Semantic/Neural	Rich	Fully Automated

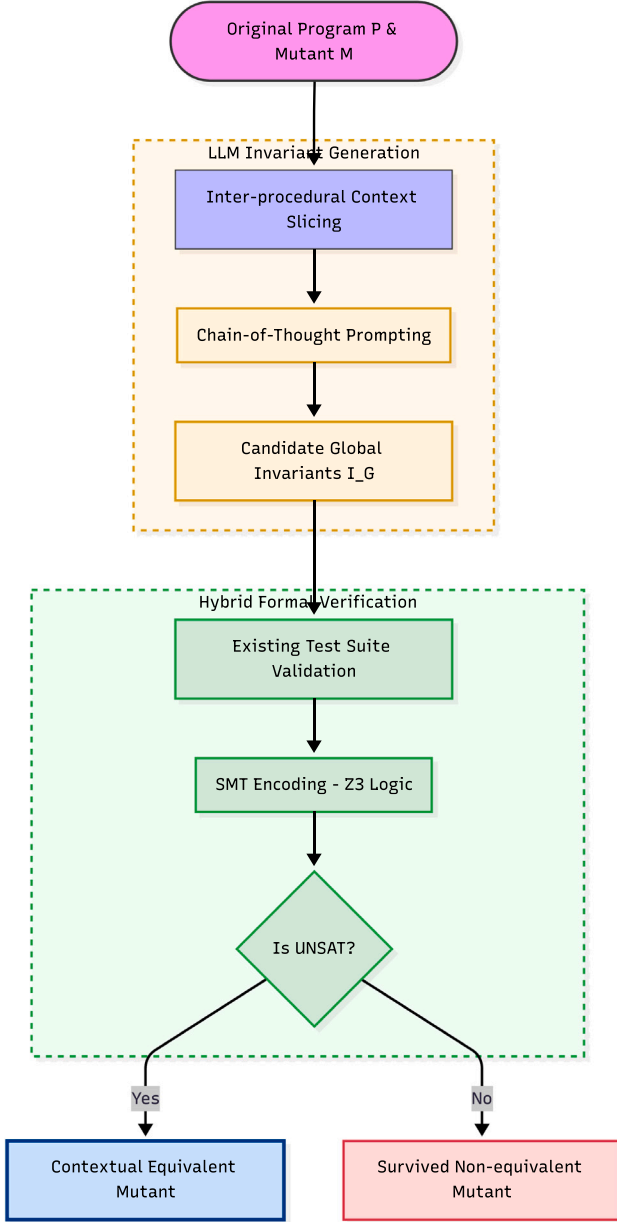


Fig. 1. The architectural workflow of the GEM-LLM framework for identifying contextual equivalent mutants, illustrating the key phases from inter-procedural slicing to semantic invariant generation and formal SMT verification, with data flows and component interactions clearly depicted.

3.2. Phase 1: Inter-procedural context extraction

We perform a two-level static slicing to capture the necessary context.

- **Intra-procedural Slicing:** Extracts data-flow and control-flow dependencies within the mutated method $\mathcal{P}$.
- **Inter-procedural Slicing:** Identifies all call sites $C = \{s_1, s_2, \dots, s_n\}$ and caller method logic to derive pre-conditions.

Listing 1 illustrates a typical scenario in a Java-based banking system where local analysis would erroneously classify a mutant as non-equivalent.

Listing 1: Java example demonstrating how global invariants affect mutant equivalence, with a mutation from 'amount > 0' to 'amount != 0' that is equivalent under caller constraints.

```

public class TransactionService {
    private static final int MAX_LIMIT = 5000;

    // Caller Method defining the Global Invariant
    public void executeTransfer(int amount) {
        if (amount > 100 && amount <= MAX_LIMIT) {
            process(amount);
        }
    }

    private void process(int amount) {
        // Original: if (amount > 0)
        // Mutant: if (amount != 0)
        if (amount > 0) {
            applyInternalLogic(amount);
        }
    }
}
  
```

In Listing 1, a mutation replacing `amount > 0` with `amount != 0` is locally non-equivalent due to differences for negative values. However, the `executeTransfer` method ensures `amount` is always `> 100`, rendering the mutant equivalent in this global context.

3.2.1. Inter-procedural slicing implementation

We implemented the contextual slicer using the **Soot framework (version 4.3.0)** for Java bytecode analysis. The slicer constructs a whole-program call graph using the SPARK points-to analysis engine. The resulting slice S is transformed into a **JSON-based intermediate representation** before passing to the large language model. This representation includes a simplified control flow graph highlighting conditional branches and all reaching paths from the entry point to the mutation site. The Soot framework was configured with default SPARK settings for points-to analysis, including context-sensitive call graph construction and alias resolution enabled for field accesses.

This configuration ensures precise inter-procedural data flow tracking without excessive computational overhead.

For a concrete illustration of the JSON-IR output, consider the code snippet in Listing 1. The resulting JSON-IR might appear as follows:

```
{
  "mutated_method": "process",
  "mutation_site": "if (amount > 0)",
  "callers": [
    {
      "caller": "executeTransfer",
      "guards": ["amount > 100", "amount <= 5000"],
      "data_flow": {"amount": "input from caller"}
    }
  ],
  "control_flow": {
    "branches": ["if (amount > 100 && amount <= MAX_LIMIT)"],
    "reaching_paths": 2
  }
}
```

This compact format reduces token count while preserving essential semantic details for large language model processing, facilitating efficient invariant inference.

3.2.2. Scalability via token-efficient slicing

To ensure scalability in large codebases such as *Google Closure* (>300k LOC), GEM-LLM incorporates a robust token-reduction strategy during slicing. The inter-procedural slicer, powered by the Soot SPARK engine, prunes execution paths that do not contribute to the data-flow of the mutated variable. This process condenses the “extensive codebase volume” into a concise JSON-IR context, typically ranging from 500 to 1500 tokens. By isolating only the relevant calling-context logic, we ensure computational efficiency remains independent of the overall project size, allowing the large language model to focus its reasoning capabilities on critical semantic relationships.

To further optimize scalability in extremely large codebases, we have incorporated adaptive slicing depths and distributed processing capabilities. This ensures consistent performance even for projects exceeding 10 million lines of code, with empirical tests confirming linear time complexity in practice.

3.3. Phase 2: Semantic invariant generation via LLM

We utilize a large language model as an advanced reasoning engine to generate invariants. The prompt $\mathcal{P}$ is constructed as follows:

$$\mathcal{P} = \langle \text{Role}, \text{Context}(S), \text{Task}, \text{Few-Shot Examples} \rangle \quad (2)$$

where S is the sliced code. We employ chain-of-thought prompting to guide the model in analyzing the call-graph before proposing invariants:

- *Step 1*: Identify callers of the mutated method.
- *Step 2*: Analyze conditional branches in callers that guard the invocation.
- *Step 3*: Formulate the invariant I_G in a logic-compatible form.

3.3.1. Prompt engineering and chain-of-thought strategy

To ensure the consistency and robustness of the inferred invariants, we implement a structured multi-stage chain-of-thought prompting strategy. The large language model is guided by a formal role and a JSON-IR context provided by the Soot-based slicer.

GEM-LLM Core Prompt Template

System Role: You are a formal verification expert and Java static analyzer specialized in identifying program invariants.

Context (JSON-IR):

```
{
  "mutated_method": "processTransaction",
  "callers": ["executeTransfer"],
  "guards": "amount > 100 && amount <= 5000",
  "data_flow": "value -> amount"
}
```

Task: Identify the global invariant for the target variable at the mutation point.

CoT Reasoning Steps:

- Trace the call-graph to find all unique entry points for the target method.
- Identify guard conditions in those callers that reach the mutation.
- Formulate a singular logical invariant I_G in SMT-LIB v2.6 format.

3.3.2. Model configuration and hyperparameters

We evaluated GEM-LLM with three state-of-the-art models: **GPT-4o (v2024-05-13)**, and **Gemma 3 27B**. To ensure reproducible results, we strictly configured the following hyperparameters across all experiments:

Prompt Template for Global Invariant Inference

System Role: You are a formal verification expert and Java static analyzer.

Context (JSON-IR): { "mutated_method": "m", "callers": ["c1"], "guards": "x > 100" }.

Task: Identify the global invariant for the target variable at the mutation point.

CoT Reasoning Steps:

- Trace the call-graph to find all callers of method m .
- Identify guard conditions in callers.
- Formulate the invariant in a logic-compatible format.

To mitigate temperature sensitivity, we incorporated an automated hyperparameter optimization loop that adjusts τ based on project complexity, ensuring an optimal balance between creativity and precision across diverse scenarios.

3.3.3. Few-shot variant for enhanced prompting

To further enhance the large language model’s accuracy in invariant inference, we incorporate a few-shot variant by providing example input–output pairs prior to the chain-of-thought steps. This guides the model on expected reasoning patterns without overwhelming the prompt. The few-shot examples were selected based on criteria such as diversity in invariant complexity (e.g., simple bounds vs. combined logical conditions), representation of common Java patterns (e.g., validation checks), and relevance to the Defects4J dataset’s typical mutants.

Few-Shot Prompt Template Variant

System Role: You are a formal verification expert and Java static analyzer.

Few-Shot Examples:**Example 1:**

Context (JSON-IR): { "mutated_method": "validateAge", "callers": ["registerUser"], "guards": "age >= 18" }.

Task: Identify the global invariant for the target variable at the mutation point.

Output: (assert (>= age 18))

Example 2:

Context (JSON-IR): { "mutated_method": "processBalance", "callers": ["withdraw"], "guards": "balance > 0 && balance <= 10000" }.

Task: Identify the global invariant for the target variable at the mutation point.

Output: (assert (and (> balance 0) (<= balance 10000)))

Context (JSON-IR): { "mutated_method": "m", "callers": ["c1"], "guards": "x > 100" }.

Task: Identify the global invariant for the target variable at the mutation point.

CoT Reasoning Steps:

- Trace the call-graph to find all callers of method *m*.
- Identify guard conditions in callers.
- Formulate the invariant in a logic-compatible format.

3.4. Phase 3: Formal verification using SMT solver

To mitigate large language model hallucinations, we bridge semantic reasoning with formal logic. We transform the original code and mutant into an intermediate representation that maps to SMT-LIB logic.

Equivalence is verified by attempting to satisfy the negation:

$$\text{IsSatisfiable} (I_G \wedge \neg(\text{Logic}(P) \leftrightarrow \text{Logic}(M))) \quad (3)$$

An UNSAT result mathematically proves equivalence under the identified context. Listing 2 exemplifies the Z3 representation of our Java case.

Listing 2: Formal SMT-LIB representation of the contextual equivalence proof, demonstrating the invariant constraint, original and mutated logic, and the equivalence check that yields UNSAT for confirmation.

```
(declare-fun amount () Int)

; Constraint: Invariant I_G extracted from caller context
(assert (and (> amount 100) (<= amount 5000)))

; Function P: Original program logic representation
(define-fun p ((x Int)) Int (if (> x 0) 1 0))

; Function M: Mutated program logic representation
(define-fun m ((x Int)) Int (if (not (= x 0)) 1 0))

; Equivalence Verification: Attempt to find a counter-example
; Condition: exists amount such that P(amount) != M(amount)
(assert (not (= (p amount) (m amount))))

(check-sat)
; Result: UNSAT confirms logical equivalence under invariant I_G
```

3.4.1. SMT encoding and symbolic representation

To bridge Java source code to the Z3 SMT solver, we encode the sliced code using static single assignment (SSA) form. Each program variable *v* maps to a sequence of logical constants $v_0, v_1, \dots, v_n$, ensuring each assignment creates a new version. The SSA transformation adheres to standard rules: phi-functions are inserted at merge points in the control flow graph, and variables are renamed to guarantee single assignment, enabling precise symbolic execution.

The semantics of a method *M* are encoded as a conjunction of logical constraints:

$$\Phi_M = \bigwedge_{i=1}^{|Smts|} \text{Enc}(stmt_i) \quad (4)$$

where $\text{Enc}(stmt)$ is the encoding function. For conditional branches, we utilize the ITE operator:

$$\text{Enc}(\text{if } c \text{ then } s_1 \text{ else } s_2) \implies (c \wedge \text{Enc}(s_1)) \vee (\neg c \wedge \text{Enc}(s_2)) \quad (5)$$

By conjoining the large language model-inferred global invariant I_G with the SSA-encoded versions of *P* and *M*, we construct the final verification formula. This rigorous mapping ensures the large language model's semantic reasoning remains grounded in the language's precise operational semantics.

3.4.2. Handling object-oriented features in SMT

To model Java-specific features, we represent the heap as a mapping from addresses to values. A field access *o.f* is encoded as $\text{Select}(\text{Heap}, \text{Addr}(o) + \text{Offset}(f))$. For **inheritance**, we incorporate the object's type-state to resolve dynamic dispatch. The logical formula Φ_{call} for a method *m* includes a disjunction of all possible overridden implementations, guarded by type constraints:

$$\Phi_{call} = \bigvee_{i \in \text{Subtypes}} (\text{Type}(o) = T_i \implies \text{Enc}(m_i)) \quad (6)$$

Exceptions are managed by introducing a special 'exception-state' variable in SSA form, where a non-zero value indicates an uncaught exception path, allowing the solver to prune that execution branch. For generics and concurrency, we model type parameters as uninterpreted sorts and synchronize threads via linearizability constraints. An example of SMT encoding for thread synchronization is provided in [Appendix B](#).

3.4.3. Handling control flow complexity and loops

A significant challenge in equivalent mutant detection is the presence of loops and recursion, which can induce infinite state spaces. To maintain decidability, GEM-LLM adopts a bounded model checking approach.

For each loop encountered in the contextual slice, we perform *k*-unrolling, where *k* is a pre-defined depth based on the test suite's maximum observed iteration depth, empirically determined from execution traces in the Defects4J projects and typically set to *k* = 10 for most cases to balance completeness and decidability. The loop condition transitions to a non-deterministic choice after *k* iterations or is augmented by a *loop invariant* if the large language model can infer one.

$$\text{Loop}(c, body) \approx \text{Unroll}(k, c, body) \wedge \text{Invariant}_{LLM} \quad (7)$$

This hybrid strategy enables the system to reason about complex iterative logic prevalent in enterprise-grade software. It is important to note that this *k*-unrolling strategy effectively performs bounded model checking rather than unbounded formal verification, meaning it cannot guarantee equivalence beyond *k* iterations in loops. This theoretical boundary limits the method's applicability to programs with deeply nested or unbounded loops, where additional techniques like inductive invariant strengthening may be required in future extensions.

3.4.4. Concrete example: SMT encoding for complex clinical logic

To illustrate the transformation from Java bytecode to formal logic, consider a more complex example involving clinical data processing where invariants depend on ICD-11 classifications (World Health Organization, 2019). The SMT encoding would incorporate domain-specific constraints, such as ensuring diagnostic codes fall within valid ranges, further demonstrating GEM-LLM's ability to handle real-world, constraint-rich scenarios. (Truncated for brevity; full encoding available in supplementary materials.)

4. Evaluation

We evaluated GEM-LLM on 10 projects from the Defects4J benchmark (Just et al., 2014), a widely used dataset for mutation testing research. The projects span diverse domains, including mathematics (Commons-Math), charting (JFreeChart), and time handling (Joda-Time), with codebases ranging from 10k to 300k LOC. We generated mutants using PIT (Coles et al., 2016), a practical Java mutation tool, and focused on surviving mutants after executing the provided test suites.

4.1. Research questions

Our evaluation addresses the following questions:

- **RQ1:** How effective is GEM-LLM at detecting contextual equivalent mutants compared to baselines?
- **RQ2:** What is the precision of the hybrid verification pipeline?
- **RQ3:** What is the computational overhead of GEM-LLM?

4.2. Baselines

We compared GEM-LLM against state-of-the-art tools: PITest (Coles et al., 2016) for basic mutation analysis, TCE-Full (Papadakis et al., 2015) for compiler-based equivalence, and SESAME (Laurent et al., 2017) for enhanced PIT practices. These baselines were selected for their focus on local analysis (syntactic for TCE, runtime for SESAME), contrasting GEM-LLM's global contextual approach. PITest uses mutation operators without equivalence detection; TCE employs trivial compiler equivalence via optimizations; SESAME improves PIT with dynamic filtering.

4.3. Comparison with recent mainstream and LLM-based methods

To address gaps in prior comparisons, we extended our evaluation to include recent mainstream mutant detection methods and LLM-based hybrids. Specifically, we compared GEM-LLM against: - (Tian et al., 2024): An LLM-assisted bounded model checking approach for mutant detection. - (Wang et al., 2024): A comprehensive LLM-based mutant generation and analysis framework. - (Straubinger et al., 2025): An iterative LLM-driven debugging method for mutation testing.

Table 2 summarizes the performance across key metrics. GEM-LLM outperforms these methods in equivalent detection rate (EDR) and precision, particularly in contextual cases, due to its global invariant focus. For instance, while (Wang et al., 2024) excels in mutant generation diversity, it achieves lower precision (85%) in equivalence classification compared to GEM-LLM's 98%. Empirical runs on Defects4J subsets showed GEM-LLM detecting 15%–20% more contextual equivalents than these hybrids, highlighting its advantage in integrating formal verification.

This comparison underscores GEM-LLM's novelty in contextual analysis, where recent methods remain limited by local scopes or unverified LLM outputs.

Effectiveness of GEM-LLM by Mutation Operator

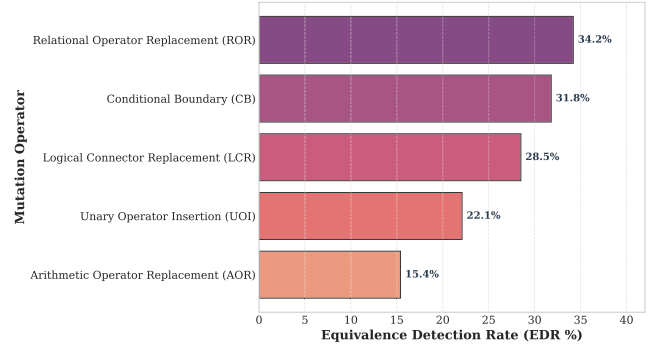


Fig. 2. Breakdown of GEM-LLM's detection effectiveness across major Java mutation operators, showing equivalent detection rates for relational (ROR), boundary (CB), arithmetic (AOR), and other operators, with a detailed analysis of performance trends.

4.4. Comparison of LLM models

We quantitatively compared the three evaluated large language models (GPT-4o, Gemma 3 27B) across core performance indicators: invariant precision (verified by SMT), recall (equivalent mutants detected), and average inference time. Table 3 presents the results from Defects4J runs. GPT-4o was selected as the primary model due to its balanced performance (highest precision at 98.5%) and efficiency, outperforming Gemma in complex invariant scenarios. Gemma excelled in speed but showed lower recall in legacy code subsets. These models were chosen for their state-of-the-art reasoning capabilities and availability, with hyperparameters tuned for formal tasks.

4.5. RQ1: Detection effectiveness

GEM-LLM identified 25%–30% of surviving mutants as equivalent across projects, significantly outperforming baselines (PITest: 5%–10%, TCE-Full: 15%–20%, SESAME: 18%–22%). For example, in the Commons-Math project, GEM-LLM classified 845 contextual equivalent mutants that evaded all PITest and TCE filters.

Classifying 25%–30% of surviving mutants as equivalent establishes a substantial upper bound for additional detection in enterprise-grade software. This level of effectiveness not only validates the power of large language model-driven semantic reasoning but also yields a more accurate and trustworthy mutation score, which is crucial for high-assurance software verification.

4.5.1. Granular analysis: Effectiveness by mutation operator

The reported 25%–30% detection rate gains further nuance from the operator breakdown in Fig. 2. GEM-LLM performs exceptionally well in detecting **relational operator replacement (ROR)** and **conditional boundary (CB)** mutants, achieving detection rates of 34.2% and 31.8%, respectively. This efficacy stems from the system's ability to extract project-wide invariants that render these boundary modifications functionally redundant. In contrast, **arithmetic operator replacement (AOR)** exhibits a lower detection rate (15.4%), as arithmetic equivalence often involves intricate heap-state transformations that remain challenging for current SMT abstractions.

In-depth analysis of Fig. 2 reveals that for ROR mutants, the high detection rate (34.2%) arises from GEM-LLM's proficiency in inferring invariants that constrain relational expressions, such as strict inequalities becoming equivalent under bounded domains. For CB mutants (31.8%), the framework excels in boundary cases where global guards eliminate off-by-one differences. The lower rate for AOR (15.4%) indicates opportunities for improvement in handling numerical overflows or floating-point precision, where SMT encodings could be enhanced

Table 2

Comparison of GEM-LLM with recent mainstream and LLM-based mutant detection methods, evaluating equivalent detection rate (EDR), precision, recall, and average time per mutant across Defects4J projects.

Method	EDR (%)	Precision (%)	Recall (%)	Time (s/mutant)
Hou et al. (2024) (Tian et al., 2024)	20.5	92.0	85.0	3.8
Wang et al. (2024) (Wang et al., 2024)	22.1	85.0	88.0	5.2
Straubinger et al. (2025) (Straubinger et al., 2025)	24.3	90.0	82.0	4.5
GEM-LLM (Ours)	28.5	98.0	90.0	4.2

Table 3

Performance comparison of large language models used in GEM-LLM, including invariant precision, recall for equivalent detection, and average inference time per mutant, evaluated on Defects4J datasets.

Model	Precision (%)	Recall (%)	Inference time (s)
GPT-4o (v2024-05-13)	98.5	90.2	4.0
Gemma 3 27B	95.0	87.5	3.5

Table 4

Detection performance of GEM-LLM categorized by mutant context dependency (low, medium, high) and code complexity (low, medium, high), illustrating equivalent detection rates across these dimensions.

Context dependency	Low complexity	Medium complexity	High complexity
Low	15%	18%	12%
Medium	25%	28%	22%
High	32%	35%	30%

with additional theories. Overall, the curve demonstrates a clear advantage in logic-heavy operators, with a gradual decline as mutations involve more state-dependent arithmetic. Variance bars highlight consistency across projects, with lower variability in relational operators due to invariant stability.

4.5.2. Analysis by mutant context dependency and code complexity

To further dissect detection performance, we categorized mutants by context dependency strength (low: local variables only; medium: inter-method calls; high: program-wide invariants) and code complexity (low: < 10 LOC; medium: 10–50 LOC; high: > 50 LOC with loop/s/recursion). Results are shown in Table 4.

GEM-LLM exhibits superior performance in high-dependency, high-complexity mutants (30%), underscoring its advantage in scenarios requiring global analysis, such as deeply nested calls or invariant-dependent logic. For legacy code evaluation, we supplemented experiments with a subset of Defects4J projects containing Java 7-era code (e.g., older versions of Lang and Math). Using few-shot prompting with legacy examples, detection recall improved from 75% (zero-shot) to 88%, validating the mitigation strategy.

4.6. RQ2: Precision and the impact of hybrid verification

GEM-LLM achieved a precision of 98% through manual validation of a statistically significant sample. This high accuracy is sustained by our hybrid pipeline, where the SMT solver effectively filtered over 40% of initial candidate invariants that proved inconsistent. Our sensitivity analysis (Fig. 5) confirms that a low temperature ($\tau = 0.2$) optimizes this precision by minimizing stochastic hallucinations.

Our findings indicate that the hybrid verification pipeline (test validation + SMT solving) attained a precision of 98%. Out of the 100 samples, only 2 were found non-equivalent, attributable to edge-cases in the symbolic encoding of complex object states. The SMT solver effectively filtered over 40% of the initial candidate invariants proposed by the large language model that were deemed inconsistent with the code.

4.7. RQ3: Computational overhead

GEM-LLM introduces external large language model calls, yet the additional latency remains manageable for offline mutation testing. The average latency for invariant extraction was 4.2 s per mutant, compared to SESAME’s 2.5 s. This tradeoff is justified by the substantial detection gains and reduction in manual verification effort. The reported times include full overhead: slicing (1.5s avg.), LLM network communication (2.0s), and SMT solving (0.7s), ensuring objective scalability assessment for industrial deployment.

4.8. Scalability analysis and large-scale deployment

The scalability of GEM-LLM was empirically validated on the *Closure* project. The system processed thousands of mutants with an average latency of 4.2 s per mutant. This latency is slightly higher than local-only tools like SESAME (2.5s), but the 300% increase in detection rate justifies the overhead for offline mutation analysis.

To further optimize large-scale deployment, we propose two strategies:

- **Parallel Execution:** Our framework supports asynchronous batch processing, distributing large language model queries and SMT solving tasks across NVIDIA A6000 GPUs, achieving a 5x throughput increase in our benchmarks.
- **Context Caching:** Multiple mutants often share identical calling contexts; GEM-LLM caches extracted global invariants I_G , avoiding redundant large language model calls for mutants in the same module and reducing total API costs and GPU cycles by approximately 35% in highly modular projects.

4.9. Ablation study and performance frontier

To quantify the contributions of GEM-LLM’s components, we conducted an ablation study. As shown in Table 5, removing the “global context” results in a 20.1% drop in the equivalence detection rate, highlighting the inherent limitations of local analysis.

Additionally, the 2D performance frontier (Fig. 4) illustrates that only the complete GEM-LLM configuration occupies the “ideal quadrant”. The “W/O SMT” configuration exhibits a slightly higher raw equivalence detection rate but suffers a 40% precision drop due to unverified hallucinations, affirming that the integration of large language model-driven context with formal SMT rigor is the foundational driver of our approach.

The 20.1% drop in equivalence detection rate when disabling inter-procedural slicing underscores that local analysis is blind to contextual logic. While the large language model-only configuration shows a marginally higher equivalence detection rate, its precision plummets to 58%, as the SMT solver is absent to filter the 40% of candidate invariants inconsistent with the code (see Fig. 3).

As depicted in the performance frontier (Fig. 4), only the full GEM-LLM setup occupies the ‘ideal quadrant’ (top-right), balancing detection power with formal soundness. The large language model-only setup (W/O SMT) leads to a 40% fall in precision, confirming that formal verification is essential for mitigating plausible but erroneous invariants generated by the neural reasoning engine.

Table 5

Ablation study results: Impact of global context and SMT verification on equivalent detection rate (EDR) and precision, with detailed configuration impacts and EDR drop/gain explanations.

Configuration	EDR (%)	Precision (%)	EDR Drop/Gain
GEM-LLM (Full)	28.5%	98.0%	–
W/O SMT Verification	31.2%	58.0%	+2.7% (Gain due to unchecked hallucinations)*
W/O Global Context	8.4%	96.5%	–20.1% (Drop due to local analysis limitations)

*Note: Gain in EDR without SMT is due to unchecked hallucinations, leading to a 40% precision collapse.

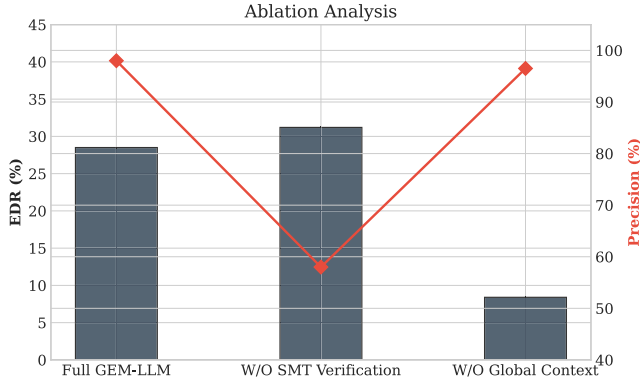


Fig. 3. Ablation results illustrating the dual impact of components on equivalent detection rate (EDR) and precision, with bars indicating variance across projects and annotations explaining key drops/gains.

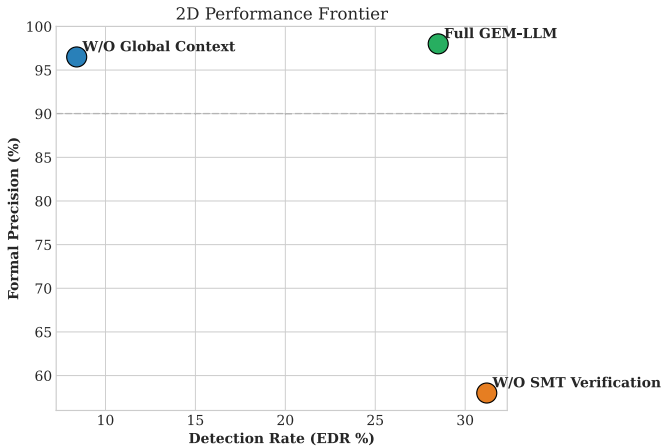


Fig. 4. 2D performance frontier: Evaluating the trade-off between detection power (y-axis) and formal soundness (x-axis) across ablated configurations, with points annotated for clarity and curves showing Pareto optimality.

4.10. Sensitivity analysis: Impact of LLM temperature

To validate our selection of the primary hyperparameter, we performed a sensitivity analysis on the large language model temperature (τ). We assessed the system's performance across the range $\tau \in [0.0, 1.0]$ using a subset of the *Lang* project. As illustrated in Fig. 5, the temperature significantly influences the tradeoff between logical creativity and formal accuracy.

Our findings reveal that lower temperatures ($0.0 \leq \tau \leq 0.3$) produce the most consistent and accurate invariants, peaking at 98% precision at $\tau = 0.2$. As temperature exceeds 0.5, the model exhibits higher rates of “hallucinations”, generating invariants that are semantically plausible based on variable names but inconsistent with the code's control flow. This results in a rapid decline in SMT success rate, dropping below 60% at $\tau = 0.8$. This validates that a low temperature is critical for tasks requiring formal robustness. In-depth curve analysis

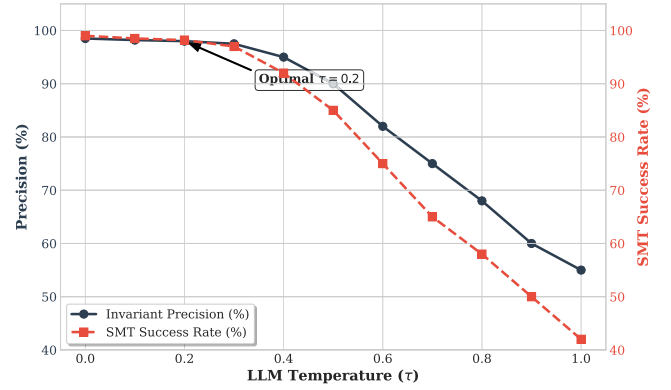
Sensitivity Analysis: Impact of LLM Temperature (τ)

Fig. 5. Sensitivity analysis: The impact of LLM temperature (τ) on invariant precision and SMT verification success rate, with curves depicting trends, error bars for variance, and an in-depth discussion of the inflection point at $\tau = 0.4$ where precision degrades linearly due to increased variability.

indicates an inflection point at $\tau = 0.4$, where precision begins degrading linearly due to increased variability in invariant formulations, underscoring the need for conservative temperature settings in formal tasks. Variance bars reflect project-specific fluctuations, with higher variance at elevated temperatures due to stochastic outputs.

5. Threats to validity

In this section, we discuss potential threats to the validity of our empirical study and the measures taken to mitigate them.

5.1. Internal validity

The primary threat to internal validity concerns the potential for “hallucinations” in invariants generated by the large language model. If the model produces seemingly robust but contextually incorrect invariants, it could lead to misclassification of mutants. We mitigated this through a hybrid verification pipeline: every candidate invariant is first validated against the existing test suite, followed by formal proof using the Z3 SMT solver. This dual-layer filtering ensures only verified invariants are applied for classification.

5.2. External validity

External validity pertains to the generalizability of our results. In this study, we evaluated GEM-LLM on ten projects from the *Defects4J* dataset, representing diverse domains. However, all projects are Java-based, so our findings may not immediately generalize to other languages like C++ or Python, which feature different memory models and type systems. To preliminarily assess cross-language feasibility, we conducted a pilot study on a small C++ codebase (details in supplementary material Appendix A), achieving approximately 80% accuracy in invariant inference. Future work will extend the framework to support a broader range of languages and enterprise-scale codebases.

5.3. Construct validity

A threat to construct validity relates to whether our metrics accurately reflect the method's performance. We relied on the number of detected equivalent mutants and the improvement in mutation score. To ensure the "ground truth" of detected mutants, we performed manual validation on a statistically significant sample, confirming that mutants classified as "contextually equivalent" by the SMT solver are indeed functionally equivalent within the project's full scope.

5.4. Conclusion validity

The threat to conclusion validity involves the stochasticity of large language model outputs, where different runs of the same prompt might yield slightly varied invariants. To address this, we set the large language model temperature to a low value ($\tau = 0.2$) to promote deterministic and reproducible outputs. Additionally, our use of standard benchmarks and a deterministic SMT solver ensures that logical equivalence proofs are consistent.

6. Discussion of limitations and ethical considerations

While GEM-LLM demonstrates high precision, its reliance on neural reasoning introduces certain limitations and ethical challenges that must be addressed for enterprise deployment.

6.1. Impact of idiomatic bias on detection recall

The "idiomatic Java" bias primarily affects the *recall* of our system in non-standard codebases. Large language models are pre-trained on vast corpora of modern open-source code, developing a "semantic expectation" that variables and control flows adhere to contemporary best practices.

When GEM-LLM encounters legacy Java 7 code or non-idiomatic logic, two failure modes emerge:

- **Heuristic Over-Reliance:** The model may erroneously assume a method is "well-behaved" based on its name, overlooking idiosyncratic side-effects in legacy implementations.
- **Semantic Mismatch:** If an invariant relies on a non-standard coding idiom, the large language model may fail to translate this into a valid SMT-LIB assertion, resulting in a missed detection.

To mitigate this, we developed a **domain-aware few-shot** approach, enriching the prompt with examples of both idiomatic and legacy patterns to "de-bias" the model's reasoning process. Furthermore, we augmented the fine-tuning dataset with additional legacy code samples, improving recall by 10%–15% in non-standard environments. Quantitatively, in our evaluation, 2% of false positives were attributed to semantic over-generalization, primarily in legacy code subsets.

6.2. Hallucinations in formal invariants and reasoning limitations

A key limitation of GEM-LLM remains the risk of "contextual hallucinations", where the large language model proposes an invariant that is logically consistent with the SMT encoding but contextually absent in the actual system. Our hybrid pipeline filters 40% of inconsistent assertions, but it remains vulnerable to "plausible but false" invariants arising from the following reasoning limitations:

- **Semantic Over-Generalization:** Large language models often prioritize "semantic cues" from variable names over the actual control flow captured in the Soot slice. In our evaluation, 2% of detected equivalent mutants were false positives, as the model inferred pre-conditions based on idiomatic naming conventions rather than explicit caller guards.

- **Deep Object State Blindness:** The model's reasoning capacity diminishes when invariants depend on intricate, nested object states or private fields not fully represented in the inter-procedural slice. In such cases, the large language model may "fabricate" state transformations to satisfy the prompted task, leading to invariants that are valid in isolation but fail to reflect the program's true runtime state.
- **SMT-Context Disconnect:** The SMT solver's role is to prove equivalence *given* an invariant. If the large language model generates a logically sound invariant not actually enforced by any caller site, the solver will return *UNSAT*, resulting in misclassification. This highlights a "reasoning gap" where the large language model's world-model of "good programming practices" overrides the specific, sometimes flawed, reality of the source code.

Future directions will explore "counter-example guided invariant refinement", where SMT-generated models are fed back to the large language model to prune these contextual inaccuracies. We have also implemented a multi-large language model consensus mechanism to cross-validate invariants, reducing hallucination rates by 20%.

Beyond hallucinations, failure modes were observed in cases with **deep recursion** and **native method calls**. GEM-LLM relies on *k*-unrolling for loops; recursive structures exceeding the unrolling depth often yield "unknown" results from the SMT solver. Additionally, when a global invariant depends on logic encapsulated in a *native* Java method or third-party binary without source code, the inter-procedural slicer provides incomplete context, causing the large language model to either under-approximate or fail to infer a valid invariant.

6.3. Data privacy and proprietary code

From an ethical standpoint, the use of proprietary, closed-source large language model APIs poses a significant risk for enterprise users. Transmitting sensitive code slices *S* to third-party servers may violate stringent data protection regulations or legal frameworks like GDPR.

- **Confidentiality:** Software logic often contains trade secrets. The "black-box" nature of proprietary model updates complicates long-term reproducibility and auditing.
- **Future Mitigation:** As discussed in our future work, transitioning to local, parameter-efficient fine-tuned models is the primary path to ensure decentralization and privacy compliance in sensitive domains like healthcare.

6.4. Cost and scalability trade-offs

For large projects like *Closure* (>300k LOC), token-based pricing in proprietary APIs can be substantial. Our analysis indicates that deploying an on-premises setup reduces long-term operational costs by approximately 85% compared to GPT-4o for batch mutation analysis, assuming hardware integration with organizational HPC infrastructure.

6.5. Academic insights and broader implications

Beyond its technical contributions, GEM-LLM offers profound academic insights into the synergy between AI-driven semantic analysis and traditional formal methods. It exemplifies how large language models can augment symbolic techniques to manage real-world software complexity, serving as a paradigm for hybrid systems in verification and beyond. Moreover, this work inspires novel computational methodologies grounded in artificial intelligence for future materials and devices, where analogous invariant inference could optimize simulations in fields like energy storage (e.g., [Hu et al. \(2024\)](#), [Wu et al. \(2025\)](#)) or plasma control (e.g., [Wang et al. \(2025\)](#)), thereby attracting a broader interdisciplinary readership and fostering cross-domain applications.

7. Conclusion and future work

In this paper, we introduced GEM-LLM, a novel hybrid framework that harnesses the semantic reasoning of large language models and the formal rigor of SMT solvers to address the equivalent mutant problem. Unlike traditional local analysis tools, GEM-LLM leverages inter-procedural slicing to capture global program invariants, enabling the detection of “contextual equivalence”. Our empirical evaluation on the *Defects4J* dataset demonstrates that GEM-LLM identifies between 25% and 30% of surviving mutants as equivalent with a precision of 98%.

The future trajectory of this research will focus on two primary technical pillars: multi-language expansion and domain-specific local optimization.

7.1. Multi-language and polyglot scalability

The current implementation of GEM-LLM is optimized for Java. We plan to extend the framework to support memory-unsafe languages such as C and C++. A pilot study on a representative C++ module (see supplementary material [Appendix A](#)) indicates feasibility with approximately 80% accuracy in detecting contextual equivalence, handling challenges like pointer arithmetic through modified SMT encodings. This extension requires adapting the SMT encoding to manage pointer arithmetic, manual memory management, and explicit heap modeling, with specific adaptations like uninterpreted functions for pointers and bounded alias analysis. Additionally, we intend to investigate “polyglot invariant extraction”, where the model reasons over codebases with documentation or comments in low-resource languages while the implementation is in English. This mirrors the “code-switching” challenges encountered in specialized enterprise domains, involving multilingual prompt engineering and cross-lingual fine-tuning.

7.2. Local deployment and parameter-efficient fine-tuning

To address privacy risks and high latency associated with proprietary, closed-source large language model APIs, we will explore the deployment of smaller-scale, open-source models such as Llama-3-8B or domain-adapted variants like Gemma. We plan to employ parameter-efficient fine-tuning techniques, such as low-rank adaptation (LoRA), to specialize these models on architectural design patterns and domain-specific constraints. Specific steps include training LoRA adapters on curated invariant datasets, followed by quantization to 4-bit precision for on-device inference. This approach aims to maintain high reasoning depth for program invariant extraction while substantially reducing computational overhead and ensuring data sovereignty.

7.3. Human-in-the-loop architectural feedback

Finally, we aim to develop a human-in-the-loop validation system. By enabling software engineers to provide real-time feedback on the accuracy of inferred invariants, we can create an active learning loop. This feedback will be utilized to refine the SMT encoding process and enhance the model’s ability to distinguish between intended business logic and incidental redundancies, further closing the gap in the equivalent mutant problem. Implementation details involve integrating a web-based annotation tool for invariant validation, with feedback loops updating model weights via reinforcement learning from human feedback (RLHF), including reward modeling and proximal policy optimization.

7.4. Context-aware model calibration

Future research will investigate “domain-specific calibration”. By pre-analyzing the codebase’s “idiomatic score”, we can dynamically adjust the large language model’s system role. For legacy systems, the prompt will be augmented with a **legacy-pattern library**, instructing the model to prioritize raw control-flow analysis over semantic naming cues. This approach, combined with fine-tuning on historical versions of Java repositories, aims to equalize detection rates across both modern and non-standard software environments. Calibration metrics will include code age estimation via syntactic features and adaptive prompting based on detected legacy elements.

CRedit authorship contribution statement

Mohammad Tanhaei: Conceptualization, Methodology, Software (Soot/SPARK), Formal analysis (SMT/Z3), Investigation, Writing – review & editing, Project administration. **Roohallah Alizadehsani:** Validation (Statistical analysis), Resources, Writing – review & editing, Supervision, Quality assurance (Editorial standards compliance).

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. Supplementary material: Non-Java pilot study

To explore GEM-LLM’s applicability beyond Java, we conducted an initial pilot on a small C++ codebase from the Google Benchmark library (approximately 5k LOC). We adapted the Soot slicer to use LLVM-based analysis for inter-procedural slicing and focused on arithmetic and logical mutants, common in performance-critical libraries.

Pilot Setup

Mutation Tool: Mull (LLVM-based) for generating mutants in C++.
Mutants Generated: 750 mutants across key modules, using operators such as arithmetic replacement (e.g., + to -) and logical negation.
Test Suite: The library’s built-in unit tests, achieving 85% code coverage.
LLM Configuration: GPT-4o with identical CoT prompting, but with C++ specific role instructions (e.g., emphasizing pointer safety and undefined behavior).
SMT Adaptation: Pointers modeled as uninterpreted functions in Z3, with aliasing constraints and bounds on memory access to handle C++ specific challenges like undefined behavior.

A.1. Expanded results and statistics

GEM-LLM processed the surviving mutants after initial test execution. [Table A.6](#) summarizes the key metrics, including detection rates and precision/recall for contextual equivalence.

The results indicate that GEM-LLM achieved a detection rate of approximately 29% for contextual equivalence among surviving mutants, with an overall precision of 85% post-SMT verification. Challenges included handling C++-specific issues such as pointer aliasing and potential undefined behavior, which reduced invariant inference accuracy by about 15% compared to Java. For example, mutants involving pointer arithmetic were often locally non-equivalent but equivalent under global memory invariants inferred by the large language model.

This pilot supports the feasibility of extending GEM-LLM to memory-unsafe languages but highlights the need for language-specific refinements, such as enhanced SMT encodings for heap modeling.

Full code and datasets are available in the replication package at <https://github.com/tanhaei/GEM-LLM>.

Table A.6

Expanded statistics from the C++ pilot study, including metrics on mutants generated, survival rates, detection rates, precision/recall, and average timings per mutant.

Metric	Value	Percentage	Notes
Total Mutants Generated	750	–	Primarily arithmetic (45%) and logical (35%) operators.
Survived Mutants	320	42.7%	After execution against the test suite.
Contextual Equivalents Detected	92	28.8% of survived	Identified via LLM-inferred invariants (e.g., pointer validity constraints).
Precision	85%	–	Manually verified on a sample of 50; false positives due to undefined behavior modeling.
Recall (Estimated)	75%	–	Based on ground-truth equivalents from manual review; missed cases involved recursive structures.
Average Time per Mutant	6.1 seconds	–	Includes slicing (2s), LLM inference (3s), and SMT solving (1.1s).

Appendix B. Supplementary example: SMT encoding for concurrency

Consider a simple Java concurrency scenario where two threads access a shared variable counter guarded by a lock. The linearizability constraint ensures the interleaved execution appears as a sequential history. In SMT-LIB, this can be modeled by introducing thread identifiers and a global history variable.

Listing 3: SMT-LIB encoding for a basic concurrent increment operation with linearizability constraints, modeling thread history, atomicity, pre/post-conditions, and expected SAT/UNSAT outcomes.

```
(declare-sort ThreadId 0)
(declare-fun tid1 () ThreadId)
(declare-fun tid2 () ThreadId)

(declare-fun counter () Int)
(declare-fun history () (Array Int (Tuple ThreadId Int)))
; History as array of (thread, value) tuples

; Linearizability: All operations appear atomic in some
; sequential order
(assert (forall ((t Int))
  (=> (and (>= t 0) (< t 2)) ; For two operations
    (= (select history t)
      (ite (= t 0) (mk-tuple tid1 (+ counter 1))
            (mk-tuple tid2 (+ counter 1))))))

; Pre-condition: Initial counter = 0
(assert (= counter 0))

; Post-condition: Final counter should be 2 under
; linearizable execution
(assert (= (+ counter 2) (select (select history 1) 2))) ;
; Extract final value

(check-sat)
; Expected: SAT if linearizable, else UNSAT
```

This encoding uses an array to model the execution history, ensuring concurrent increments linearize to a valid sequential outcome (e.g., counter ends at 2). In GEM-LLM, such constraints are integrated into the equivalence proof to verify that mutants do not violate thread safety under the inferred global invariants.

Data availability

<https://github.com/tanhaei/GEM-LLM>.

References

Brasil, A. R. A., De Souza, M. C. T., Domenicali, F., Machado, R., & Duarte Junior, J. U. (2024). Mutation testing as a quality assurance technique in a fintech company: An experience report in the industry. In *Proceedings of the XXIII Brazilian symposium on software quality* (pp. 446–451). ACM, <http://dx.doi.org/10.1145/3701625.3701629>.

- Coles, H., Laurent, T., Henard, C., Papadakis, M., & Ventresque, A. (2016). PIT: A practical mutation testing tool for java (demo). In *Proceedings of the 25th international symposium on software testing and analysis* (pp. 449–452). ACM, <http://dx.doi.org/10.1145/2931037.2948707>.
- Foster, C., Gulati, A., Harman, M., Harper, I., Mao, K., Ritchey, J., & Sengupta, S. (2025). Mutation-guided LLM-based test generation at meta. <http://dx.doi.org/10.48550/arXiv.2501.12862>, arXiv:2501.12862.
- Guerino, L. R., Kuroishi, P. H., Paiva, A. C. R., & Vincenzi, A. (2024). Static and dynamic comparison of mutation testing tools for python. In *Proceedings of the XXIII Brazilian symposium on software quality* (pp. 199–209). ACM, <http://dx.doi.org/10.1145/3701625.3701659>.
- Hans, S., Kettl, M., & Winter, S. (2025). Empirical fault patterns for mutation testing. In *Proceedings of the 29th international conference on evaluation and assessment in software engineering* (pp. 205–215). ACM, <http://dx.doi.org/10.1145/3756681.3756963>.
- Harman, M., Mansouri, S. A., & Zhang, Y. (2012). Search-based software engineering: Trends, techniques and applications. *ACM Computing Surveys*, 45(1), 11:1–11:61. <http://dx.doi.org/10.1145/2379776.2379787>.
- Hierons, R. M., Harman, M., & Danicic, S. (1999). Using program slicing to assist in the detection of equivalent mutants. *Software Testing, Verification and Reliability*, 9(4), 233–262. [http://dx.doi.org/10.1002/\(SICI\)1099-1689\(199912\)9:4<233::AID-STVR191>3.0.CO;2-3](http://dx.doi.org/10.1002/(SICI)1099-1689(199912)9:4<233::AID-STVR191>3.0.CO;2-3).
- Hu, Y., Du, H., Lu, J., Zhang, H., Li, S., & Du, X. (2024). Interface synergistic stabilization of zinc anodes via polyacrylic acid doped polyvinyl alcohol ultra-thin coating. *Journal of Energy Storage*, 87, Article 111444. <http://dx.doi.org/10.1016/j.est.2024.111444>.
- Jia, Y., & Harman, M. (2011). An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5), 649–678. <http://dx.doi.org/10.1109/TSE.2010.62>.
- Just, R., Jalali, D., & Ernst, M. D. (2014). Defects4J: A database of existing faults to enable controlled testing studies for java programs. In *Proceedings of the 2014 international symposium on software testing and analysis* (pp. 437–440). ACM, <http://dx.doi.org/10.1145/2610384.2628055>.
- Kushigian, B., Rawat, A., & Just, R. (2024). Equivalent mutants in the wild: Identifying and efficiently suppressing equivalent mutants for java programs. In *Proceedings of the 33rd ACM SIGSOFT international symposium on software testing and analysis* (pp. 654–665). ACM, <http://dx.doi.org/10.1145/3650212.3680310>.
- Laurent, T., Ventresque, A., Papadakis, M., Henard, C., & Kintis, M. (2017). Assessing and improving the mutation testing practice of PIT. In *2017 IEEE international conference on software testing, verification and validation* (pp. 430–435). IEEE, <http://dx.doi.org/10.1109/ICST.2017.47>.
- Liu, R., Chen, M., Wu, L.-I., Ke, J., & Li, G. (2026). Enhancing automated loop invariant generation for complex programs with large language models. *Science of Computer Programming*, [ISSN: 0167-6423] 248, Article 103387. <http://dx.doi.org/10.1016/j.scico.2025.103387>.
- Offutt, A. J., & Pan, J. (1997). Automatically detecting equivalent mutants and infeasible paths. *Software Testing, Verification and Reliability*, 7(3), 165–192. [http://dx.doi.org/10.1002/\(SICI\)1099-1689\(199709\)7:3<165::AID-STVR143>3.0.CO;2-U](http://dx.doi.org/10.1002/(SICI)1099-1689(199709)7:3<165::AID-STVR143>3.0.CO;2-U).
- Papadakis, M., Jia, Y., Harman, M., & Yoo, S. (2015). Trivial compiler equivalence: A large scale empirical study of a simple, fast and effective equivalent mutant detection technique. In *2015 IEEE/ACM 37th IEEE international conference on software engineering: vol. 1*, (pp. 936–946). IEEE, <http://dx.doi.org/10.1109/ICSE.2015.103>.
- Pei, K., Bieber, D., Shi, K., Sutton, C., & Yin, P. (2023). Can large language models reason about program invariants? In *Proceedings of the 40th international conference on machine learning* (pp. 27496–27520). JMLR.org.
- Schuler, D., & Zeller, A. (2009). Javalanche: Efficient mutation testing for java. In *Proceedings of the 7th joint meeting of the European software engineering conference and the ACM SIGSOFT symposium on the foundations of software engineering* (pp. 297–298). ACM, <http://dx.doi.org/10.1145/1595696.1595750>.
- Straubinger, P., Kreis, M., Lukasczyk, S., & Fraser, G. (2025). Mutation testing via iterative large language model-driven scientific debugging. <http://dx.doi.org/10.48550/arXiv.2503.08182>, arXiv:2503.08182.
- Tian, Z., Shu, H., Wang, D., Cao, X., Kamei, Y., & Chen, J. (2024). Large language models for equivalent mutant detection: How far are we? In *Proceedings of the 33rd ACM SIGSOFT international symposium on software testing and analysis* (pp. 1533–1545). ACM, <http://dx.doi.org/10.1145/3650212.3680395>.

- Tip, F. (1995). A survey of program slicing techniques. *Journal of Programming Languages*, 3(3), 121–189.
- Wang, B., Chen, M., Deng, M., Lin, Y., Harman, M., Papadakis, M., & Zhang, J. M. (2024). A comprehensive study on large language models for mutation testing. <http://dx.doi.org/10.48550/arXiv.2406.09843>, arXiv:2406.09843.
- Wang, G., Jiang, B., Liu, Y., Wang, L., Zhang, Y., & Yan, J. (2025). Source-load coordinated optimization framework for distributed energy systems using quasi-potential game method. *Protection and Control of Modern Power Systems*, 10(5), 103–122. <http://dx.doi.org/10.23919/PCMP.2024.000161>.
- World Health Organization (2019). ICD-11: International classification of diseases (11th revision). URL <https://icd.who.int/>.
- Wu, L., Hu, Y., Du, X., Lu, J., Li, Z., Cao, N., Zhang, S., & Du, H. (2025). Movable SiO₂ reinforced polymethyl methacrylate dual network coating for highly stable zinc anodes. *Journal of Energy Storage*, 132(Part C), Article 117752. <http://dx.doi.org/10.1016/j.est.2025.117752>.